大　老　二

Big Two Card Game

學　　號	411485026
**姓　　**	吳威佑
程式語言	C++
作業類型	大老二遊戲實作
遊戲人數	2 ～4 人
檔案數量	6

♠ ♡ ♢ ♣

Contents

1 程式概述 **4**
1.1 遊戲規則簡述 4
1.2 檔案結構 4

2 類別設計與資料結構 **4**
2.1 Poker 類別（card.h） 4
2.1.1 牌值計算邏輯 — `takevalue()` 5
2.1.2 顯示字串轉換 — `getnutw()` 6
2.1.3 Unicode 撲克牌符號顯示 — `displaycard()` 6
2.2 Player 類別（player.h） 7

3 函式設計（func.cpp） **8**
3.1 牌型判斷 — `getPatternType()` 8
3.2 移除手牌 — `removeCard()` 9
3.3 顯示桌面牌 — `showTableCards()` 10
3.4 顯示玩家手牌 — `showPlayerHand()` 10

4 主程式邏輯（411485026_Bigtwo.cpp） **11**
4.1 啟用 Windows ANSI 彩色輸出 11
4.2 遊戲初始化流程 12
4.3 初始化牌組與洗牌 12
4.4 發牌特例：三人模式 13
4.5 理牌 13
4.6 HUD 儀表板設計 14
4.7 玩家輸入處理 15
4.8 Sleep 延遲機制 16
4.9 出牌驗證邏輯 17
4.10 Pass 與桌面清空機制 18

5 AI 電腦邏輯 **19**
5.1 AI 策略概述 19
5.2 AI 尋牌流程 19
5.3 AI 找順子邏輯 19
5.4 AI 找葫蘆邏輯 20
5.5 AI 找鐵支邏輯 21
5.6 AI 找同花順邏輯 22
5.7 AI 出牌與記錄 22

6 計分與排行榜 **24**
6.1 計分規則 24
6.2 最後一名自動結算 24
6.3 最終排行榜 25

7 一些細節 **26**
7.1 為何需要 `numusedtocal`？ 26
7.2 印牌多行同時顯示 27
7.3 歷史紀錄 `continue` 技巧 27
7.4 排行榜用指標陣列排序 27
7.5 錯誤處理與輸入驗證 27

8 程式碼結果 **28**
8.1 已知問題 28

9 成品展示 **28**
9.1 遊戲大廳介面 28
9.2 HUD 儀表板 29
9.3 歷史紀錄 29
9.4 最終排行榜 30

1. 程式概述

以 C++ 實作標準大老二遊戲，支援 2 到 4 位玩家，其中玩家 1 由真人操控，其餘由電腦 AI 控制。支援多局制，每局結束後累積分數，並在所有局結束後顯示最終排行榜。

1.1 遊戲規則簡述

- 使用標準 52 張撲克牌，3 最小、2 最大；花色大小為 ♠ > ♡ > ♢ > ♣
- 持有**梅花 3（♣3）**的玩家優先出牌，且第一手必須包含此牌
- 出牌型態：**單張**、**一對**、**五張牌（順子 < 葫蘆 < 鐵支 < 同花順）**
- 不可出牌則 Pass；若其他所有玩家均 Pass，桌面清空，最後出牌者自由出牌
- 最先出完手牌者得分最高（依完牌順序給分），最後剩牌者得分最低

1.2 檔案結構

檔案	類型	功能
411485026_Bigtwo.cpp	主程式	遊戲主迴圈、玩家輸入、AI 邏輯
card.h / card.cpp	牌類別	牌的屬性、牌值計算、顯示
player.h	玩家類別	玩家狀態記錄（無.cpp）
func.h / func.cpp	工具函式	牌型判斷、顯示手牌、刪除牌

2. 類別設計與資料結構

2.1 Poker 類別（card.h）

```cpp
class Poker {
public:
    int num;              // 原始牌號 (1=A, 2~10, 11=J, 12=Q, 13=K)
    int numusedtocal;     // 計算用數值 (3->0 最小, 2->12 最大)
    std::string numusedtowatch; // 顯示用字串 (1->A, 11->J, 12->Q,
        13->K)
    std::string color;    // 花色 ("S", "H", "D", "C")
    int value;            // 保留欄位

    Poker();
    int takevalue();      // 回傳牌的絕對值 (0~51)
    std::string getnutw(); // 取得顯示字串
    void displaycard(int line);
};
```

Listing 1: card.h — Poker 類別宣告

2.1.1 牌值計算邏輯 — takevalue()

牌值計算公式

大老二規則：3 最小、2 最大；花色 ♠ > ♡ > ♢ > ♣
步驟一：將牌號映射至 0~12

$$\text{numusedtocal} = \begin{cases} \text{num} - 3 & \text{若 num} \geq 3 \\ \text{num} + 10 & \text{若 num} < 3 \end{cases}$$

步驟二：計算絕對值（0~51）

$$\text{value} = \text{numusedtocal} \times 4 + \text{colorOffset}$$

其中 colorOffset:

$$\spadesuit \to 3, \quad \heartsuit \to 2, \quad \diamondsuit \to 1, \quad \clubsuit \to 0$$

結果：梅花 3 = 0（最小），黑桃 2 = 51（最大），共 52 個唯一值。

```cpp
int Poker::takevalue() {
    int val = 0;
    if (num >= 3) {
        numusedtocal = num - 3;   // 3->0, 4->1, ..., K->10
    }
    else {
        numusedtocal = num + 10;  // A->11, 2->12
    }

    val += (numusedtocal * 4);    // 乘以 4 是因為每個號數有 4 種花色

    if (color == "S") {           // 黑桃最大
        val += 3;
    }
    else if (color == "H") {      // 紅心第二
        val += 2;
    }
    else if (color == "D") {      // 方塊第三
        val += 1;
    }
    else if (color == "C") {      // 梅花最小
        val += 0;
    }
    return val;
}
```

Listing 2: card.cpp — takevalue()

下表列出各牌號對應的 `numusedtocal` 值：

牌號	3	4	5	6	7	8	9	10	J	Q	K	A	2
numusedtocal	0	1	2	3	4	5	6	7	8	9	10	11	12

2.1.2 顯示字串轉換 — getnutw()

```
string Poker::getnutw() {
    if (num == 1) {
        numusedtowatch = "A";
    }
    else if (num == 11) {
        numusedtowatch = "J";
    }
    else if (num == 12) {
        numusedtowatch = "Q";
    }
    else if (num == 13) {
        numusedtowatch = "K";
    }
    else {
        numusedtowatch = std::to_string(num);  // 2~10 直接轉字串
    }
    return numusedtowatch;
}
```

Listing 3: 將牌號轉為顯示用字串

2.1.3 Unicode 撲克牌符號顯示 — displaycard()

UTF-8 編碼處理

為了顯示真正的撲克牌符號（♠♥♦♣）與框線（┌┐│└┘），程式使用了 Unicode 字元，為了正常顯示，加入以下代碼：

```
#ifdef _MSC_VER
#pragma execution_character_set("utf-8")
#endif
```

這確保原始碼中的 UTF-8 字元能正確編譯與執行。

```
void Poker::displaycard(int line) {
    std::string suitStr = "";
    std::string colorCode = "";
    std::string resetCode = "\033[0m";

    // 將花色代碼轉為撲克牌符號，並設定顏色
    if (color == "S") {
        suitStr = "♠";                          // 黑桃
    }
    else if (color == "H") {
        suitStr = "♥"; colorCode = "\033[31m";  // 紅心 + 紅色
    }
    else if (color == "D") {
        suitStr = "♦"; colorCode = "\033[31m";  // 方塊 + 紅色
    }
    else if (color == "C") {
        suitStr = "♣";                          // 梅花
    }

    std::cout << colorCode;  // 在印出每一行之前，先套用顏色
```

```
    std::string nutw = getnutw();

    switch (line) {
    case 0:
        std::cout << "┌─────────┐";  // 頂部框線
        break;
    case 1:
        // 左上角顯示花色和牌號，根據牌號長度調整間距
        if (nutw.length() == 1) std::cout << "│ " << suitStr << nutw << "     │";
        else                    std::cout << "│ " << suitStr << nutw << "    │";
        break;
    case 2:
        std::cout << "│         │";
        break;
    case 3:
        std::cout << "│    " << suitStr << "    │";  // 中央花色
        break;
    case 4:
        std::cout << "│         │";
        break;
    case 5:
        // 右下角顯示牌號和花色，根據牌號長度調整間距
        if (nutw.length() == 1) std::cout << "│     " << nutw << suitStr << " │";
        else                    std::cout << "│    " << nutw << suitStr << " │";
        break;
    case 6:
        std::cout << "└─────────┘";  // 底部框線
        break;
    }

    std::cout << resetCode;  // 恢復預設顏色
}
```

Listing 4: displaycard() — 使用 Unicode 符號

彩色輸出

`\033[31m` 將後續文字設為紅色。`\033[0m` 重置所有格式。
在主程式啟動時需要啟用 Windows 虛擬終端處理（第 4.1 節）。

2.2 Player 類別（player.h）

```
class Player {
public:
    int id;
    int totscore;       // 累積總分
    int cardcount;      // 手牌數量
    int lastRoundRank;  // 本局完牌排名（用於同分判斷勝負）
};
```

Listing 5: player.h — Player 類別宣告

`player.cpp` 檔案被移除，因為 Player 類別沒有需要實作的成員函式。所有玩家狀態都是簡單的資料欄位。

3. 函式設計（func.cpp）

3.1 牌型判斷 — getPatternType()

遊戲邏輯的函式，根據選牌數量與牌面辨識出合法的五張牌型。

```
int getPatternType(vector<Poker>& hand) {
    int n = hand.size();
    if (n == 1) {
        return 1;  // 單張
    }
    if (n == 2 && hand[0].num == hand[1].num) {
        return 2;  // 一對
    }
    if (n == 5) {
        // 順子 同花順
        bool isStraight = true;
        bool isFlush = true;

        // 檢查是否連號
        for (int i = 1; i < 5; i++) {
            if (hand[i].numusedtocal != hand[i - 1].numusedtocal + 1) {
                isStraight = false;
            }
            if (hand[i].color != hand[0].color) {
                isFlush = false;
            }
        }

        // 特例：A-2-3-4-5 (numusedtocal: 0,1,2,11,12)
        // 因為排序後是 3-4-5-A-2，不符合連續遞增，需強制承認
        bool isA2345 = (hand[0].numusedtocal == 0 && hand[1].numusedtocal == 1 &&
            hand[2].numusedtocal == 2 && hand[3].numusedtocal == 11 &&
            hand[4].numusedtocal == 12);

        if (isA2345) {
            isStraight = true; // 強制承認它是順子
        }

        // 排除JQKA2 (numusedtocal: 8,9,10,11,12)
        if (isStraight && hand[0].numusedtocal == 8) {
            isStraight = false;
        }

        if (isStraight && isFlush) {
            return 6;  // 同花順
        }
        if (isStraight) {
```

```
            return 3;  // 順子
        }

        // 判定鐵支
        if (hand[0].num == hand[3].num || hand[1].num == hand[4].num) {
            return 5;  // 鐵支
        }

        // 葫蘆
        if ((hand[0].num == hand[2].num && hand[3].num == hand[4].num) ||
            (hand[0].num == hand[1].num && hand[2].num == hand[4].num)) {
            return 4;  // 葫蘆
        }
    }
    return 0;  // 無效組合
}
```

Listing 6: getPatternType()

牌型回傳值對照表

回傳值	牌型	大小比較基準	說明
1	單張	唯一牌值	52 個唯一值直接比較
2	一對	較大那張的牌值	手牌排序後取 [1]
3	順子	最大張牌值	手牌排序後取 [4]
4	葫蘆	三條的牌值	手牌排序後取 [2]
5	鐵支	鐵支的牌值	手牌排序後取 [2]
6	同花順	最大張牌值	手牌排序後取 [4]
0	無效	—	不合法組合

特殊處理：A-2-3-4-5 順子

由於 `numusedtocal` 的排列是 3=0 到 K=10，A=11，2=12，排序後的 A-2-3-4-5 手牌順序為 **[3, 4, 5, A, 2]**，即 `numusedtocal = [0, 1, 2, 11, 12]`，不連續，因此需要額外以 `isA2345` 強制設為順子。
同時排除 J-Q-K-A-2（`hand[0].numusedtocal == 8`）。

3.2 移除手牌 — `removeCard()`

```
void removeCard(int index, Poker hand[40], int& handCount) {
    // 將 index 後的所有牌左移一格
    for (int i = index; i < handCount - 1; i++) {
        hand[i] = hand[i + 1];
    }
```

```
    // 清空最後一個位置
    hand[handCount - 1] = Poker();
    handCount--;  // 手牌數量減一
}
```

Listing 7: removeCard()

多索引刪牌注意事項

重要：出多張牌時，先對索引**由大到小排序**，再逐一呼叫 `removeCard()`。
原因：刪除索引 3 的牌後，原本索引 5 的牌變成索引 4，若先刪小索引，會導致後續刪除位置錯誤。
主程式中使用以下方式確保順序：

```
sort(indices.begin(), indices.end(), greater<int>());
for (int i = 0; i < indices.size(); i++) {
    removeCard(indices[i], playerc[currentPlayer], handCounts[
        currentPlayer]);
}
```

3.3 顯示桌面牌 — showTableCards()

```
void showTableCards(Poker tableCards[], int cardCount) {
    std::cout << "\n=table
        ==========================================================" <<
         std::endl;

    if (cardCount == 0) {
        std::cout << std::endl << std::endl << std::endl << std::endl
                  << std::endl << std::endl << std::endl;
        std::cout << "
            ==========================================================
            table=\n" << std::endl;
        return;
    }

    // 印出桌面上的牌 (每張牌 7 行)
    for (int line = 0; line < 7; line++) {
        for (int i = 0; i < cardCount; i++) {
            tableCards[i].displaycard(line);
            std::cout << "   ";
        }
        std::cout << std::endl;
    }
    std::cout << "
        ==========================================================
        table=\n" << std::endl;
}
```

Listing 8: showTableCards()

3.4 顯示玩家手牌 — showPlayerHand()

```cpp
void showPlayerHand(Poker hand[], int cardCount) {
    if (cardCount == 0) {
        return;
    }

    int cardsPerRow = 6;  // 每排最多印幾張
    int totalRows = (cardCount + cardsPerRow - 1) / cardsPerRow;  // 總共需要幾排

    // 外層迴圈：控制現在印到第幾排
    for (int r = 0; r < totalRows; r++) {
        int startIdx = r * cardsPerRow;
        int endIdx = std::min(startIdx + cardsPerRow, cardCount);

        // 印出這一排的所有牌（7 行）
        for (int line = 0; line < 7; line++) {
            for (int i = startIdx; i < endIdx; i++) {
                hand[i].displaycard(line);
                std::cout << "  ";
            }
            std::cout << std::endl;
        }

        // 印出這排牌的索引標籤（對不齊...）
        for (int i = startIdx; i < endIdx; i++) {
            if (i < 10) {
                std::cout << "   [" << i << "]       ";
            }
            else {
                std::cout << "   [" << i << "]      ";
            }
        }
        std::cout << "\n\n";
    }
}
```

Listing 9: showPlayerHand()

多行並排顯示原理

每張牌佔 7 行（line 0~6）。為讓多張牌「並排」顯示而非上下堆疊，使用**雙層迴圈**：
外層：控制現在印 line 0~6 的哪一行
內層：對這一排的每張牌，印出該行的內容
這樣每個 line 印完才換行，視覺上形成多張牌並排的效果。
手牌超過 6 張時，使用**分排顯示**，每排最多 6 張，避免自動換行破壞排版。

4. 主程式邏輯（411485026_Bigtwo.cpp）

4.1 啟用 Windows ANSI 彩色輸出

```cpp
// --- 解鎖 Windows 終端機的彩色顯示功能
```

```
HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
DWORD dwMode = 0;
GetConsoleMode(hOut, &dwMode);
dwMode |= 0x0004; // ENABLE_VIRTUAL_TERMINAL_PROCESSING 的底層代碼
SetConsoleMode(hOut, dwMode);
//--------------------------------------------------反正跟紅色牌有關
```

Listing 10: 主程式開頭 — 啟用虛擬終端機處理

4.2 遊戲初始化流程

1. **設定**：玩家輸入人數（2~4）與遊玩局數，有設錯誤處理與輸入驗證 (防呆)
2. **建立牌組**：依序生成 52 張牌（S/H/D/C × A~K），並計算 `numusedtocal`
3. **洗牌**：`rand() % 52`
4. **發牌**：2/4 人各得 26/13 張；3 人各得 17 張，剩 1 張補給有梅花 3 的玩家
5. **理牌**：對每位玩家的手牌以 `takevalue()` 排序，由小到大排列

4.3 初始化牌組與洗牌

```
// 初始化牌組
for (int j = 0; j < 4; j++) {
    for (int i = 0; i < 13; i++) {
        if (j == 0) {
            deck[j * 13 + i].color = "S";  // 黑桃
        }
        else if (j == 1) {
            deck[j * 13 + i].color = "H";  // 紅心
        }
        else if (j == 2) {
            deck[j * 13 + i].color = "D";  // 方塊
        }
        else if (j == 3) {
            deck[j * 13 + i].color = "C";  // 梅花
        }
        deck[j * 13 + i].num = i + 1;  // 1~13 (A~K)
    }
}

// 讓3最小，2最大 -> 用 numusedtocal 比大小
for (int i = 0; i < 52; i++) {
    if (deck[i].num >= 3) {
        deck[i].numusedtocal = deck[i].num - 3;
    }
    else {
        deck[i].numusedtocal = deck[i].num + 10;
    }
}
```

Listing 11: 初始化牌組 — 生成 52 張牌

```
// 洗牌
for (int i = 0; i < 52; i++) {
    int r = rand() % 52;
```

```
    swap(deck[i], deck[r]);  // std::swap
}
```

Listing 12: 隨機洗牌

4.4 發牌特例：三人模式

```
if (playercount != 3) {
    // 2 人或 4 人：平均發牌
    int k = 0;
    for (int i = 0; i < playercount; i++) {
        for (int j = 0; j < 52 / playercount; j++) {
            playerc[i][j] = deck[k];

            // 找梅花三玩家
            if (playerc[i][j].numusedtocal == 0 && playerc[i][j].color
                == "C") {
                c3p = i;
            }

            k++;
            handCounts[i]++;
        }
    }
}
else {
    // 三人發牌：每人17張，剩下一張給有梅花三的
    int k = 0;
    for (int i = 0; i < playercount; i++) {
        for (int j = 0; j < 51 / playercount; j++) {  // 51 / 3 = 17
            playerc[i][j] = deck[k];
            k++;
            handCounts[i]++;
        }
    }

    // 找到持有梅花三的玩家
    for (int i = 0; i < playercount; i++) {
        for (int j = 0; j < 51 / playercount; j++) {
            if (playerc[i][j].numusedtocal == 0 && playerc[i][j].color
                == "C") {
                c3p = i;
                break;
            }
        }
    }

    // 將第 52 張牌補給有梅花三的玩家
    playerc[c3p][17] = deck[51];
    handCounts[c3p]++;  // c3p 玩家共 18 張
}
```

Listing 13: 三人發牌 — 17+17+18 張

4.5 理牌

```
// 理牌
for (int pID = 0; pID < playercount; pID++) {
    for (int i = 0; i < handCounts[pID] - 1; i++) {
        for (int j = i + 1; j < handCounts[pID]; j++) {
            // 若前面的牌比後面大，交換
            if (playerc[pID][i].takevalue() > playerc[pID][j].takevalue()) {
                Poker temp = playerc[pID][i];
                playerc[pID][i] = playerc[pID][j];
                playerc[pID][j] = temp;
            }
        }
    }
}
```

Listing 14: 為每位玩家的手牌排序

4.6 HUD 儀表板設計

每回合開始前，程式先呼叫 `clearScreen()` 清畫面，再重新繪製完整的遊戲狀態：

```
// ================== HUD 儀表板繪製開始 ==================
clearScreen();
cout << "\n
    ==================================================================\n
    ";
cout << "                          大 老 二   B I G   T W O
                        \n";
cout << "
    ==================================================================\n
    ";

// 印出所有玩家剩餘牌數
for (int i = 0; i < playercount; i++) {
    if (i == 0) {
        cout << "[ 你(玩家1) 剩 " << handCounts[i] << " 張 ]    ";
    }
    else {
        cout << "[ 玩家" << i + 1 << " 剩 " << handCounts[i] << " 張 ]
                ";
    }
}
cout << "\n
    ------------------------------------------------------------------\n
    ";

if (tableType == 0) {
    cout << "【 桌面目前為空，請自由出牌 】\n";
}
else {
    cout << "【 桌面牌型：由 玩家" << lastPlayPlayer + 1 << " 打出 】\
        n";
}

// 畫出公共牌桌
showTableCards(tableCards, currentTableCardCount);
```

```
// ================== HUD 儀表板繪製結束 ==================
```

Listing 15: HUD 儀表板繪製

4.7 玩家輸入處理

```
if (currentPlayer == 0) {  // 玩家 1（真人）
    showPlayerHand(playerc[currentPlayer], handCounts[currentPlayer]);
    cout << endl;
    cout << "請輸入索引(如 0 2 4)、輸入 -1 Pass、輸入 -2 歷史、輸入 -3 規則說明、輸入 -4 投降: ";

    // 讀取整行輸入
    string input;
    getline(cin >> ws, input);  // ws 清除前導空白
    stringstream ss(input);
    vector<int> indices;
    int idx;
    while (ss >> idx) {
        indices.push_back(idx);
    }
    sort(indices.begin(), indices.end());  // 由小到大排序

    // -2 = 查看歷史紀錄
    if (indices.size() == 1 && indices[0] == -2) {
        clearScreen();
        cout << "=歷史紀錄 ==========================================================\n\n";
        if (actionLog.empty()) {
            cout << "  (目前尚無紀錄)\n";
        }
        else {
            for (int i = 0; i < actionLog.size(); i++) {
                cout << actionLog[i] << endl;
            }
        }
        cout << "\n ==========================================================\n";

        system("pause");  // 暫停畫面，按任意鍵繼續

        continue;
    }

    // -3 = 查 看 遊 戲 說 明
    if (indices.size() == 1 && indices[0] == -3) {
    clearScreen();
    cout << "\n ==========================================================\n";
    cout << "                    大 老 二   遊 戲 說 明                    \n";
    cout << " ==========================================================\n\n";
```

```

    // ... 說明 ...
    "==================================================================\n"
        ;

    system("pause");

    continue;
}

// -4 = 投 降 彩 蛋
if (indices.size() == 1 && indices[0] == -4) {
    // ... 彩蛋 ...
}

    // ... 處理出牌邏輯 ...
}
```

Listing 16: 玩家輸入解析

continue：不破壞迴圈的查詢

輸入 -2 後，程式顯示歷史紀錄並執行 continue，跳過後續的出牌邏輯，直接回到 while(!gameOver) 迴圈頂端。
由於 currentPlayer 未改變，仍然輪到玩家 1 操作，HUD 重新繪製後玩家可以繼續出牌。讓查看歷史不消耗回合，可以隨時查閱再決定。

4.8 Sleep 延遲機制

```
#include <thread>
#include <chrono>

using std::this_thread::sleep_for;
using std::chrono::milliseconds;

// 在遊戲各處使用
std::this_thread::sleep_for(std::chrono::milliseconds(1500));
```

Listing 17: 使用 std::this_thread::sleep_for 延遲

Sleep 延遲的使用時機

程式中多處使用 sleep_for 讓遊戲節奏更自然：

- **遊戲準備階段**（1500ms）：顯示「正在準備牌局，請稍候...」後暫停
- **錯誤提示**（1000ms）：顯示「牌型不合或點數不夠大！」等警告後暫停，確保玩家看清楚訊息
- **AI 出牌後**（1500ms）：電腦出牌後暫停，讓玩家觀察桌面變化
- **完牌提示**（1000ms）：有玩家出完牌時暫停

這些延遲提升遊戲的閱讀性跟真實性，也解決電腦出太快看不到的問題。

4.9 出牌驗證邏輯

```
if (indices.size() == 1) {  // 出一張

    if (tableType == 0 || (tableType == 1 && playerc[currentPlayer][indices[0]].takevalue() > tableValue)) {
        // 第一手必須出梅花三
        if (firstroundfirsthand == true && playerc[currentPlayer][indices[0]].takevalue() != 0) {
            cout << "第一手必須出梅花三！" << endl;
            std::this_thread::sleep_for(std::chrono::milliseconds(1000));
            continue;
        }
        else {
            cout << "你出了一張牌: " << playerc[currentPlayer][indices[0]].color
                << "-" << playerc[currentPlayer][indices[0]].getnutw() << endl;

            // 更新牌桌狀態
            firstroundfirsthand = false;
            lastPlayPlayer = currentPlayer;
            tableType = 1;
            tableValue = playerc[currentPlayer][indices[0]].takevalue();
            passCount = 0;
            tableCards[0] = playerc[currentPlayer][indices[0]];
            currentTableCardCount = 1;
            removeCard(indices[0], playerc[currentPlayer], handCounts[currentPlayer]);
        }
    }
    else {
        cout << "牌型不合或點數不夠大！" << endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(1000));
        continue;
    }
}
```

Listing 18: 出牌合法性檢查示例 — 單張牌

出牌合法性判斷規則

條件	說明
桌面為空	tableType == 0，任何合法牌型均可出
單張對單張	tableType == 1，出牌值 > tableValue
一對對一對	tableType == 2，較大牌值 > tableValue
五張牌型	tableType >= 3，牌型 > 桌面牌型，或同牌型且最大值 > 桌面最大值
第一手限制	firstroundfirsthand == true，必須包含梅花 3，且不可 Pass

4.10 Pass 與桌面清空機制

```
// finishedPlayers = 完牌玩家數
int finishedPlayers = 0;
for (int i = 0; i < playercount; i++) {
    if (hasscored[i]) {
        finishedPlayers++;
    }
}

// 剩餘玩家 = playercount - finishedPlayers
// 若 passCount = (剩餘活躍玩家 - 1)，表示其他人全 pass
if (passCount >= (playercount - finishedPlayers - 1)) {
    actionLog.push_back("--- 所有對手皆 pass，桌面清空！輪到玩家 " +
        std::to_string(currentPlayer + 1) + " 自由出牌 ---");

    // 桌面歸零
    tableType = 0;
    tableValue = -1;
    passCount = 0;
    currentTableCardCount = 0;

    // 清空桌面牌陣列
    for (int i = 0; i < 5; i++) {
        tableCards[i] = Poker();
    }

    cout << "============================" << endl;
    cout << "所有對手皆 pass，桌面清空！" << endl;
    cout << "============================" << endl;
}
```

Listing 19: 連續 Pass 後桌面清空

Pass 計數的跳過機制

使用 `hasscored[]` 標記已完牌的玩家。在計算剩餘玩家時，已完牌的玩家不計入。
輪到下一位玩家時，使用以下跳過已完牌者：

```
currentPlayer = (currentPlayer + 1) % playercount;
while (hasscored[currentPlayer] == true) {
    currentPlayer = (currentPlayer + 1) % playercount;
}
```

確保完牌玩家不被要求出牌，也不影響 Pass 數。

5. AI 電腦邏輯

5.1 AI 策略概述

用迴圈找分排行去找手牌中所有能出牌的可能，並將每個可能當作一個選項，放進 vector 裡面，再用隨機抽籤的方式，讓電腦的選擇更隨機，比較不會那麼死板。

5.2 AI 尋牌流程

1. **桌面為空** → 隨機出牌
2. **桌面為單張** → 找最小的單張出
3. **桌面為一對** → 掃描相鄰同號的兩張牌，找最小能壓的一對
4. **五張牌處理**（依優先序嘗試）：
   (a) 嘗試找**順子**：掃描每張牌，以其為起點找連號 5 張，排除 JQKA2
   (b) 嘗試找**葫蘆**：掃描三條組合，配對剩餘的對子
   (c) 嘗試找**鐵支**：掃描連續四張同號
   (d) 嘗試找**同花順**：掃描同花色連號 5 張
5. **無法出牌** → Pass

5.3 AI 找順子邏輯

```
// 嘗試找順子
for (int i = 0; i < handCounts[currentPlayer] - 4; i++) {
    int startNum = playerc[currentPlayer][i].numusedtocal;
    int expectedNum = startNum + 1;
    vector<int> tempIdx = {i};

    // 從下一張開始找連號
    for (int j = i + 1; j < handCounts[currentPlayer]; j++) {
        if (playerc[currentPlayer][j].numusedtocal == expectedNum) {
            tempIdx.push_back(j);
            expectedNum++;
            if (tempIdx.size() == 5) {
                break;  // 找到 5 張連號
```

```
            }
        }
    }

    if (tempIdx.size() == 5) {
        // 排除 JQKA2 (numusedtocal: 8,9,10,11,12)
        if (playerc[currentPlayer][tempIdx[0]].numusedtocal != 8) {
            int maxVal = playerc[currentPlayer][tempIdx[4]].takevalue();

            // 檢查是否能壓過桌面
            if (tableType == 0 || tableType < 3 ||
               (tableType == 3 && maxVal > tableValue)) {
                aiPlayIndices = tempIdx;
                break;
            }
        }
    }
}
```

Listing 20: AI 尋找順子的實作

5.4 AI 找葫蘆邏輯

```
// 嘗試找葫蘆
for (int i = 0; i < handCounts[currentPlayer] - 2; i++) {
    // 找三條
    if (playerc[currentPlayer][i].num == playerc[currentPlayer][i + 1].num &&
        playerc[currentPlayer][i].num == playerc[currentPlayer][i + 2].num) {

        vector<int> tempIdx = {i, i + 1, i + 2};

        // 在其他位置找對子
        for (int j = 0; j < handCounts[currentPlayer] - 1; j++) {
            if (j == i || j == i + 1 || j == i + 2) continue;  // 跳過三條本身

            if (playerc[currentPlayer][j].num == playerc[currentPlayer][j + 1].num) {
                tempIdx.push_back(j);
                tempIdx.push_back(j + 1);

                sort(tempIdx.begin(), tempIdx.end());  // 排序索引

                vector<Poker> selected;
                for (int idx : tempIdx) {
                    selected.push_back(playerc[currentPlayer][idx]);
                }

                int pattern = getPatternType(selected);
                if (pattern == 4) {  // 確認是葫蘆
                    int midVal = selected[2].takevalue();

                    // 檢查是否能壓過桌面
                    if (tableType == 0 || tableType < 4 ||
```

```
                        (tableType == 4 && midVal > tableValue)) {
                         aiPlayIndices = tempIdx;
                         break;
                    }
                }

                tempIdx.pop_back();
                tempIdx.pop_back();
            }
        }

        if (!aiPlayIndices.empty()) break;
    }
}
```

Listing 21: AI 尋找葫蘆的實作

5.5 AI 找鐵支邏輯

```
// 嘗試找鐵支
for (int i = 0; i < handCounts[currentPlayer] - 3; i++) {
    // 找四張同號
    if (playerc[currentPlayer][i].num == playerc[currentPlayer][i + 1].num &&
        playerc[currentPlayer][i].num == playerc[currentPlayer][i + 2].num &&
        playerc[currentPlayer][i].num == playerc[currentPlayer][i + 3].num) {

        vector<int> tempIdx = {i, i + 1, i + 2, i + 3};

        // 找任意一張配牌（鐵支 + 單張）
        if (i > 0) {
            tempIdx.insert(tempIdx.begin(), i - 1);  // 前面的牌
        }
        else if (i + 4 < handCounts[currentPlayer]) {
            tempIdx.push_back(i + 4);  // 後面的牌
        }
        else {
            continue;  // 沒有配牌可用
        }

        sort(tempIdx.begin(), tempIdx.end());

        // 建立選牌並判斷牌型
        vector<Poker> selected;
        for (int idx : tempIdx) {
            selected.push_back(playerc[currentPlayer][idx]);
        }

        int pattern = getPatternType(selected);
        if (pattern == 5) {  // 確認是鐵支
            int midVal = selected[2].takevalue();

            // 檢查是否能壓過桌面
            if (tableType == 0 || tableType < 5 ||
                (tableType == 5 && midVal > tableValue)) {
```

```
                    aiPlayIndices = tempIdx;
                    break;
                }
            }
        }
}
```

Listing 22: AI 尋找鐵支的實作

5.6 AI 找同花順邏輯

```
// 嘗試找同花順
for (int i = 0; i < handCounts[currentPlayer] - 4; i++) {
    int startNum = playerc[currentPlayer][i].numusedtocal;
    string targetColor = playerc[currentPlayer][i].color;
    int expectedNum = startNum + 1;
    vector<int> tempIdx = {i};

    // 從下一張開始找連號且同花色
    for (int j = i + 1; j < handCounts[currentPlayer]; j++) {
        // 條件 1.連號 2.花色完全一樣
        if (playerc[currentPlayer][j].numusedtocal == expectedNum &&
            playerc[currentPlayer][j].color == targetColor) {
            tempIdx.push_back(j);
            expectedNum++;
            if (tempIdx.size() == 5) {
                break;
            }
        }
    }

    if (tempIdx.size() == 5) {
        // 排除JQKA2
        if (playerc[currentPlayer][tempIdx[0]].numusedtocal != 8) {
            int maxVal = playerc[currentPlayer][tempIdx[4]].takevalue
                ();

            // 檢查是否能壓過桌面
            if (tableType == 0 || tableType < 6 ||
               (tableType == 6 && maxVal > tableValue)) {
                aiPlayIndices = tempIdx;
                break;
            }
        }
    }
}
```

Listing 23: AI 尋找同花順的實作

5.7 AI 出牌與記錄

```
if (aiPlayIndices.empty()) {
    // AI pass
    actionLog.push_back("玩家 " + std::to_string(currentPlayer + 1) +
        " (電腦) : Pass");
    passCount++;
```

```
    cout << "玩家 " << currentPlayer + 1 << " (電腦) 選擇 pass" << endl;
    cout << "------------------------------------------------------------------------" << endl;
}
else {
    // AI 出牌 (以五張牌為例)
    if (aiPlayIndices.size() == 5) {
        vector<Poker> selected;
        for (int idx : aiPlayIndices) {
            selected.push_back(playerc[currentPlayer][idx]);
        }
        int pattern = getPatternType(selected);

        cout << "玩家 " << currentPlayer + 1 << " (電腦) 出了 ";
        if (pattern == 3) cout << "順子: ";
        else if (pattern == 4) cout << "葫蘆: ";
        else if (pattern == 5) cout << "鐵支: ";
        else if (pattern == 6) cout << "同花順: ";

        for (int idx : aiPlayIndices) {
            cout << playerc[currentPlayer][idx].color << "-"
                 << playerc[currentPlayer][idx].getnutw() << " ";
        }
        cout << endl;

        // 更新遊戲狀態
        firstroundfirsthand = false;
        lastPlayPlayer = currentPlayer;
        tableType = pattern;

        // 根據牌型決定比較值
        if (pattern == 4 || pattern == 5) {
            tableValue = selected[2].takevalue();  // 葫蘆/鐵支看中間牌
        }
        else {
            tableValue = selected[4].takevalue();  // 順子/同花順看最大牌
        }

        passCount = 0;

        // 更新桌面牌
        for (int i = 0; i < 5; i++) {
            tableCards[i] = selected[i];
        }
        currentTableCardCount = 5;

        // 記錄到歷史
        string detail = "";
        for (int idx : aiPlayIndices) {
            detail += playerc[currentPlayer][idx].color + "-"
                   + playerc[currentPlayer][idx].getnutw() + " ";
        }
        actionLog.push_back("玩家 " + std::to_string(currentPlayer + 1)
```

```
                                + " (電腦) : 五張牌 (" + detail + ")");

            // 從大到小排序，以免移除牌時影響未移除的牌位置
            sort(aiPlayIndices.begin(), aiPlayIndices.end(), greater<int>());
            for (int i = 0; i < aiPlayIndices.size(); i++) {
                removeCard(aiPlayIndices[i], playerc[currentPlayer], handCounts[currentPlayer]);
            }
        }
        cout << "------------------
```

```cpp
int finishedPlayers = 0;
for (int i = 0; i < playercount; i++) {
    if (hasscored[i]) {
        finishedPlayers++;
    }
}

// 只剩一位玩家未完牌時，自動結算
if (finishedPlayers == playercount - 1) {
    for (int i = 0; i < playercount; i++) {
        if (hasscored[i] == false) {
            cout << "玩家 " << i + 1 << " 是最後一名，本局結束！" << endl;
            player[i].totscore += curpoint;

            player[i].lastRoundRank = currentRank;

            cout << "玩家 " << i + 1 << " 獲得 " << curpoint << " 分！" << endl;
            hasscored[i] = true;
        }
    }
    gameOver = true;  // 結束本局

    std::this_thread::sleep_for(std::chrono::milliseconds(1000));
    continue;
}
```

Listing 26: 剩餘一人時自動結束本局

6.3 最終排行榜

```cpp
clearScreen();
cout << "\n ==================================================================\n";
cout << "                    遊 戲 結 束 - 最 終 排 行 榜                    \n";
cout << " ==================================================================\n\n";

// 建立指標陣列
Player* rankBoard[4];
for (int i = 0; i < playercount; i++) rankBoard[i] = &player[i];

// 總分高的在前；同分時，lastRoundRank 小（完牌早）的優先
for (int i = 0; i < playercount - 1; i++) {
    for (int j = 0; j < playercount - i - 1; j++) {
        if (rankBoard[j]->totscore < rankBoard[j + 1]->totscore ||
            (rankBoard[j]->totscore == rankBoard[j + 1]->totscore &&
             rankBoard[j]->lastRoundRank > rankBoard[j + 1]->lastRoundRank)) {

            Player* temp = rankBoard[j];
            rankBoard[j] = rankBoard[j + 1];
```

```
                rankBoard[j + 1] = temp;
            }
        }
    }

    // 印出排名
    for (int i = 0; i < playercount; i++) {
        // 還原玩家 ID
        int originalID = 0;
        for (int k = 0; k < playercount; k++) {
            if (rankBoard[i] == &player[k]) {
                originalID = k + 1;
                break;
            }
        }
        cout << "  第 " << i + 1 << " 名： 玩家 " << originalID
             << " (總分: " << rankBoard[i]->totscore << " 分)\n";
    }
    cout << "\n
        ==================================================================\n
        ";
    system("pause");
```

Listing 27: 最終排行榜排序與顯示

使用指標陣列排序

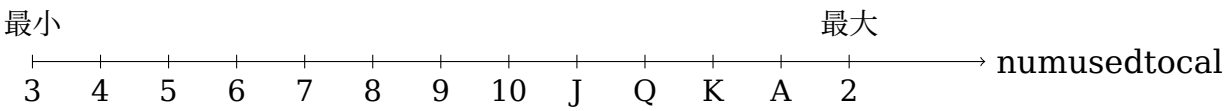
直接對 `Player` 物件陣列排序會打亂索引，導致玩家順序亂掉。
改用 `Player* rankBoard[]`，排序時只會移動到指標，最後透過**指標位址比對**回原始陣列索引：

```
if (rankBoard[i] == &player[k]) {
    originalID = k + 1;  // 還原原始玩家編號
    break;
}
```

7. 一些細節

7.1 為何需要 numusedtocal？

牌號 1（A）到 13（K）按數字排列，但大老二規則是 3 最小、2 最大，無法直接以 num 比較大小。numusedtocal 將牌號重新排序至 0~12：



這樣比較和排序可直接操作整數，簡化判斷邏輯。

7.2 印牌多行同時顯示

每張牌佔 7 行（line 0~6）。為讓多張牌「並排」顯示，`showPlayerHand()` 使用**雙層迴圈**：

- **外層迴圈**：控制 line（0~6），即現在要印哪一行
- **內層迴圈**：對這一排的每張牌，都印出該 line 的內容

這樣每個 line 印完才換行，形成視覺上的並排效果。

當手牌超過 6 張時，使用**分排顯示**（`cardsPerRow = 6`），每排最多 6 張，避免終端機自動換行破壞排版。

7.3 歷史紀錄 continue 技巧

```
if (indices[0] == -2) {
    system("pause");
    continue;   // 跳回 while(!gameOver) 頂端
                // currentPlayer 沒有變->回合不變
}
```

用 `continue` 的特性跳過回合推進邏輯，讓玩家可以在查看歷史後，回到相同狀態繼續出牌。

7.4 排行榜用指標陣列排序

直接對 `Player` 物件陣列排序會打亂索引，導致無法對應原始玩家 ID。改用 `Player* rankBoard[]`，排序時只移動指標，最後再透過指標位址比對回原始索引，同時兼顧排序效率與 ID 保留。

7.5 錯誤處理與輸入驗證

```
while (true) {
    cout << "  請輸入遊玩人數 (2~4 人): ";

    if (!(cin >> playercount)) {
        cin.clear();              // 清除錯誤旗標
        cin.ignore(10000, '\n');  // 清除輸入緩衝區
        cout << "\n  [錯誤] 請輸入有效的數字！\n\n";
        system("pause");
        continue;
    }

    if (playercount >= 2 && playercount <= 4) {
        break;
    }
    cout << "\n  [錯誤] 玩家人數必須在 2 到 4 之間！\n\n";
    system("pause");
}
```

Listing 28: 遊戲大廳輸入驗證

用 `cin.clear()` 清除錯誤旗標，`cin.ignore()` 清除緩衝區，避免進入無限迴圈。

8. 程式碼結果

8.1 已知問題

限制與改進空間

- **Windows 才能跑**：用了 `<windows.h>` 和 `system("cls")`，僅能在 Windows 環境下正常執行。若要跨平台，需改用 `<ncurses.h>`。
- **UTF-8 編碼**：在非 UTF-8 環境，撲克牌符號會變成亂碼。
- **索引對齊**：當手牌索引達到兩位數時，索引標籤的間距會略有不齊。

9. 成品展示

9.1 遊戲大廳介面

```
==================================================================
                    大 老 二   B I G   T W O
==================================================================

  請輸入遊玩人數 (2~4 人): 2

==================================================================
                    大 老 二   B I G   T W O
==================================================================

  已選擇模式: 2 人對戰

  請選擇遊玩局數 (例如 1, 3, 5): 3

==================================================================
                    大 老 二   B I G   T W O
==================================================================

  [ 遊戲設定完成 ]
  玩家人數 : 2 人
  遊玩局數 : 3 局

  正在準備牌局，請稍候...

==================================================================
```

9.2 HUD 儀表板

```
==================================================================
                    大 老 二   B I G   T W O
==================================================================
[ 你(玩家1) 剩 13 張 ]    [ 玩家2 剩 13 張 ]
------------------------------------------------------------------
【 桌面牌型：由 玩家2 打出 】

=table===========================================================
┌──────┐  ┌──────┐
│♠3    │  │♥K    │
│      │  │      │
│  ♠   │  │  ♥   │
│      │  │      │
│    3♠│  │    K♥│
└──────┘  └──────┘
==================================================================table=

輪到玩家 1 出牌
┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐
│♣3    │ │♦5    │ │♠6    │ │♥7    │ │♦9    │ │♣10   │
│      │ │      │ │      │ │      │ │      │ │      │
│  ♣   │ │  ♦   │ │  ♠   │ │  ♥   │ │  ♦   │ │  ♣   │
│      │ │      │ │      │ │      │ │      │ │      │
│    3♣│ │    5♦│ │    6♠│ │    7♥│ │    9♦│ │   10♣│
└──────┘ └──────┘ └──────┘ └──────┘ └──────┘ └──────┘
  [0]      [1]      [2]      [3]      [4]      [5]

請輸入出的牌索引(如: 0 2 4)、輸入 -1 Pass、輸入 -2 查看歷史紀錄:
```

9.3 歷史紀錄

```
=歷史紀錄=========================================================

玩家 1 : 單張 C-3
玩家 2 (電腦) : 單張 H-K
玩家 1 : Pass
--- 所有對手皆 pass，桌面清空！輪到玩家 2 自由出牌 ---
玩家 2 (電腦) : 一對 D-4 / H-4
玩家 1 : 一對 S-6 / H-6

==================================================================
```

9.4 最終排行榜

```
==================================================================
                    遊 戲 結 束  -  最 終 排 行 榜
==================================================================

  第 1 名： 玩家 1 (總分: 11 分)
  第 2 名： 玩家 2 (總分: 7 分)

==================================================================
```

♠ ♡ ♢ ♣